

L2 - 13.6. Dynamic Analysis: Debugging

Task 1 - Introduction

Malware authors understand that the malware will be analyzed and want to thwart it. This can be done through various evasion techniques. To defeat these evasion techniques, a malware analyst desires more control over malware execution

Debugging is the process of executing a program instruction-by-instruction within a controlled environment (a debugger) to inspect its memory, CPU registers, and dynamic behavior in real time. While software developers debug to fix broken code, security professionals debug for **offensive analysis**, **defense**, and **vulnerability research**.

Task 2 - The Need for Advanced Dynamic Analysis

Malware analysts keep devising new techniques to analyse malware, while malware authors devise new techniques to evade detection. Malware often hides or tries to look like legitimate software to evade the prying eyes of a malware analyst. Because we are not executing the malware during static analysis, the main focus of evading static analysis is to obfuscate the true functionality of the program until the program is executed. The following techniques can be commonly used to achieve this.

- **Changing the hash:** Slightly altering malware code—such as adding a NOP instruction—changes its unique file hash, allowing it to easily bypass basic hash-based detection mechanisms.
- **Defeating AV signatures:** Malware authors defeat signature-based antivirus detection by obfuscating code and altering static byte patterns to evade signature matching.
- **Obfuscation of strings:** Malware authors obfuscate critical strings like C2 domains, decoding them only at runtime to prevent static string searches from revealing their infrastructure.

- **Runtime loading of DLLs:** Malware uses `LoadLibrary` to dynamically load DLLs at runtime, hiding its API dependencies from static PE header analysis.
- **Packing and Obfuscation:** Malware packers wrap malicious code to block static analysis, unpacking and executing the actual payload in memory only at runtime.

Evasion of Basic Dynamic Analysis: Malware authors use environment-detection techniques to determine if their code is running in a controlled analysis setup, altering its behavior to evade dynamic detection. The following techniques are commonly used for this purpose:

- **Identification of VMs:** Malware detects virtualized environments by checking for VM-specific registry keys, drivers, or low system resources, altering its behavior to avoid revealing malicious activity during dynamic analysis.
- **Timing attacks:** Malware uses extended sleep delays and execution timing comparisons to outwait or detect automated analysis sandboxes that manipulate time.
- **Traces of user activity:** Malware checks for signs of human presence—such as mouse movement, browser history, and recent files—and executes benignly if these traces are missing to evade detection in automated sandboxes.
- **Identification of analysis tools:** Malware scans active process lists and open window titles for known security tools—like ProcMon or OllyDbg—and switches to a benign execution path if detected.

Answer the questions below

Malware sometimes checks the time before and after the execution of certain instructions to find out if it is being analysed. What type of analysis technique is bypassed by this attack?

basic dynamic analysis

What is a popular technique used by malware authors to obfuscate malware code from static analysis and unwrap it at runtime?

Packing

Task 3 - Introduction to Debugging

Debugging a malware sample trying to evade detection or reverse engineering can also be considered a program having a bug. A malware reverse engineer often has to debug a program to remove any roadblocks that prevent it from performing its malicious activity. Therefore, interactive debugging becomes an essential part of advanced malware analysis. Debuggers provide a malware analyst with the control desired to monitor a running program more closely, looking at the changes in different registers, variables, and memory regions as each instruction is executed. A debugger also allows a malware analyst to change the variables' values to control the program's flow at runtime, providing greater control over the execution path the malware follows.

Types of Debuggers:

1. Source-Level Debuggers: Source-level debuggers operate directly on high-level source code, allowing developers to track variables and logic flaws during software development.

2. Assembly-Level Debuggers: Assembly-level debuggers analyze compiled binaries without source code, allowing malware analysts to inspect CPU registers, memory states, and execution flow.

3. Kernel-Level Debuggers: Kernel-level debuggers operate directly on the operating system core, requiring a dual-system setup to analyze low-level drivers and kernel routines without freezing the host machine.

Answer the questions below

Can we recover the pre-compilation code of a compiled binary for debugging purposes?

no

Which type of debugger is used for debugging compiled binaries?

assembly-level debugger

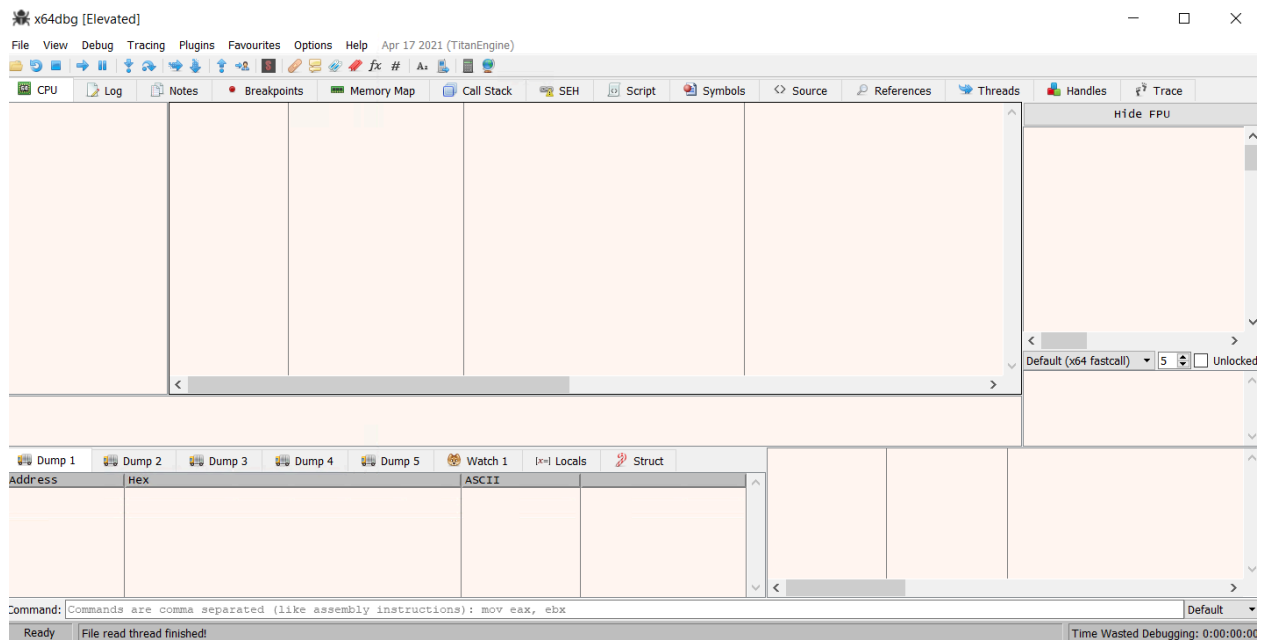
Which debugger works at the lowest level among the discussed debuggers?

Kernel-level debugger

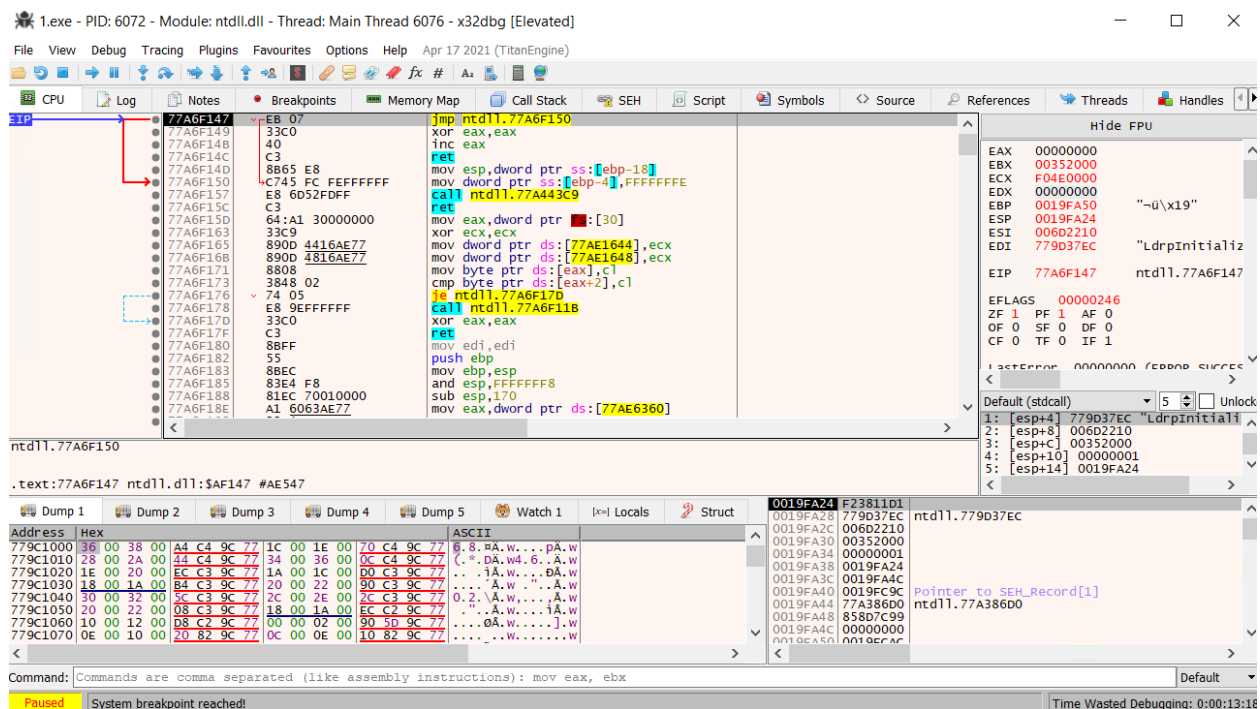
Task 4 - Familiarization With a Debugger

For malware analysis, there are many options to choose a debugger from. These options include Windbg, Ollydbg, IDA, and Ghidra. For this room, we will be using x32dbg and x64dbg.

The attached VM is essentially a FLARE VM, distributed by Mandiant (previously FireEye, now part of Google), widely used for reverse engineering malware. To start, we can navigate to Desktop > Tools > debuggers > x32dbg.exe to run the debugger.

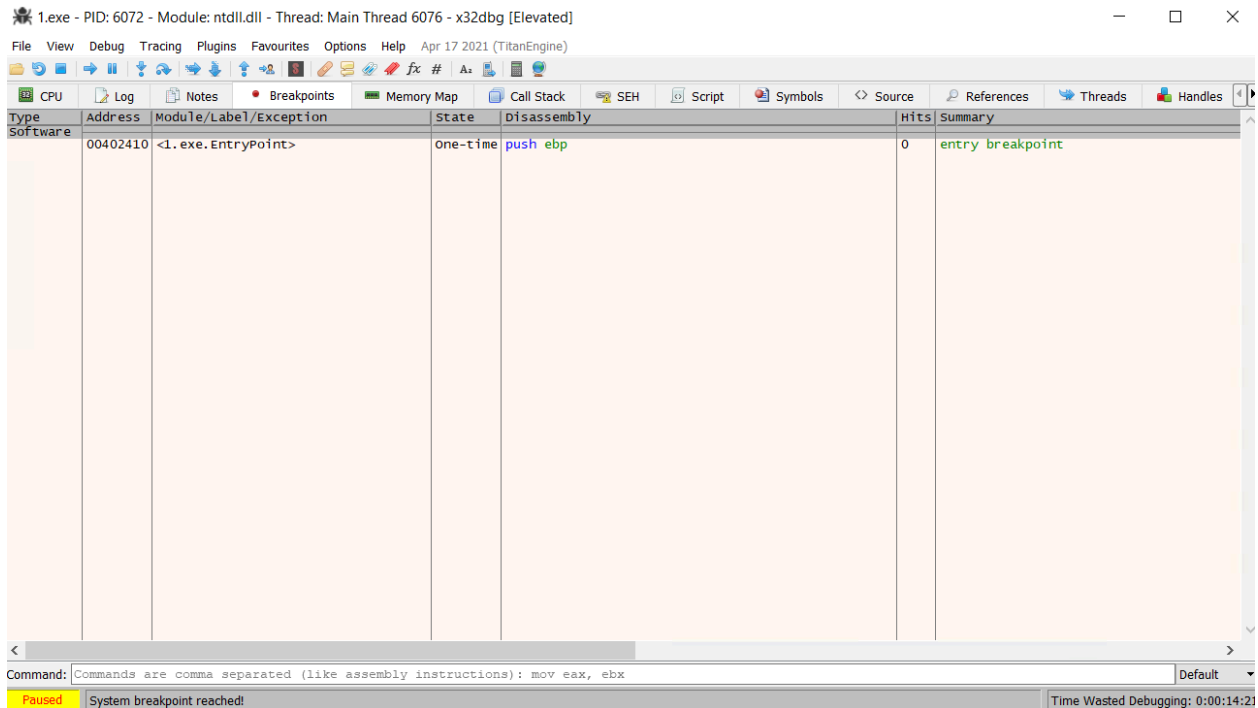


To open a file in the debugger, we can navigate to File > Open and open our desired file.

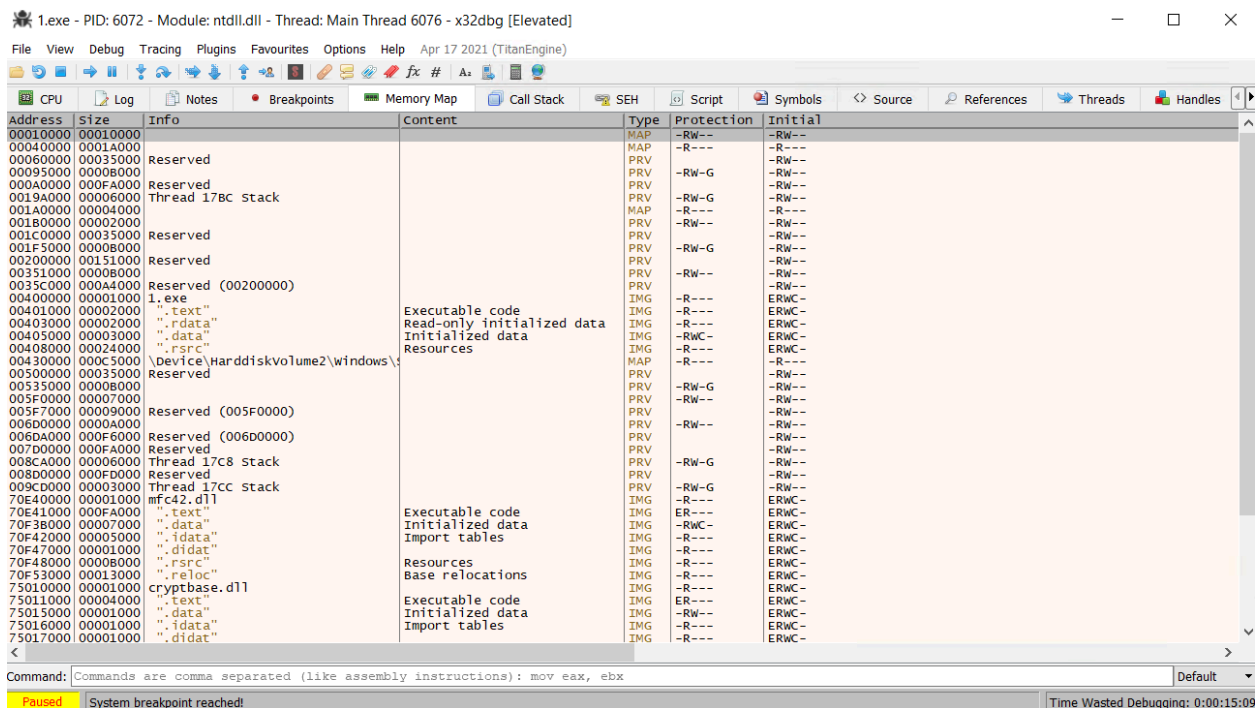


As we can see in the bottom left corner, the execution of the program is paused because a System breakpoint has been reached. In the right pane, we can see the registers and their values. We can note that the value in EIP is the address EIP (Extended Instruction Pointer) is pointing to in the disassembly pane.

The breakpoints tab shows the current status of breakpoints. Breakpoints are points where the execution of the program is paused for the analyst to analyze the registers and memory. A breakpoint on a specific instruction can be enabled by clicking the dot in front of that instruction in the CPU tab.



The Memory Map shows the memory of the program.



We can also see the Call stack of the program.

1.exe - PID: 6072 - Module: ntdll.dll - Thread: Main Thread 6076 - x32dbg [Elevated]

File View Debug Tracing Plugins Favourites Options Help Apr 17 2021 (TitanEngine)

CPU Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles

Thread	Address	To	From	Size	Comment	Party
6076	0019FA54	77A698E0	77A6F147	25C	ntdll.77A6F147	System
	0019FCB0	77A25257	77A698E0	58	ntdll.77A698E0	System
	0019FD08	77A25151	77A25257	C	ntdll.77A25257	System
	0019FD14	00000000	77A25151		ntdll.77A25151	User
6092	009CFF74	750C0419	77A3216C	10	ntdll.77A3216C	System
	009CFF84	77A2662D	750C0419	5C	kernel32.750C0419	System
	009CFFE0	77A265FD	77A2662D	10	ntdll.77A2662D	System
	009CFFF0	00000000	77A265FD		ntdll.77A265FD	User
6088	008CFF74	750C0419	77A3216C	10	ntdll.77A3216C	System
	008CFF84	77A2662D	750C0419	5C	kernel32.750C0419	System
	008CFFE0	77A265FD	77A2662D	10	ntdll.77A2662D	System
	008CFFF0	00000000	77A265FD		ntdll.77A265FD	User

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:00:15:39

The threads running in the program are shown in the threads tab.

1.exe - PID: 6072 - Module: ntdll.dll - Thread: Main Thread 6076 - x32dbg [Elevated]

File View Debug Tracing Plugins Favourites Options Help Apr 17 2021 (TitanEngine)

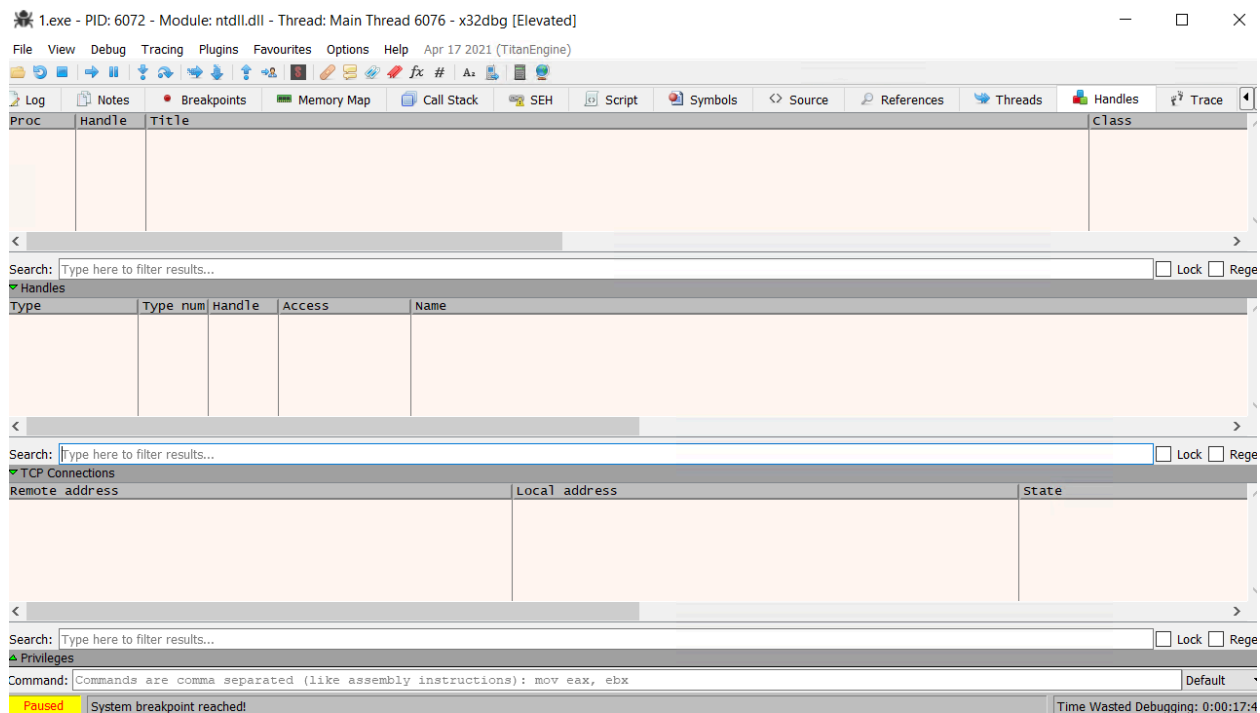
Log Notes Breakpoints Memory Map Call Stack SEH Script Symbols Source References Threads Handles Trace

Number	ID	Entry	TEB	EIP	Suspend Count	Priority	Wait Reason	Last Error	User Time	Kernel Time	Creation Time	CPU Cycles
2	6092	77A0E230	0035B000	77A3216C	1	Normal	Suspended	00000000	00:00:00.0156250	00:00:00.0000000	05:21:14.6289380	C7294C
main	6076	00402410	00355000	77A6F147	1	Normal	Executive	00000000	00:00:00.0000000	00:00:00.0312500	05:21:14.5392866	69411FF
1	6088	77A0E230	00358000	77A3216C	1	Normal	Suspended	00000000	00:00:00.0000000	00:00:00.0312500	05:21:14.6103455	F4E820

Command: Commands are comma separated (like assembly instructions): mov eax, ebx

Paused System breakpoint reached! Time Wasted Debugging: 0:00:17:00

Any handles to files, processes, or other resources the process accesses are shown in the handles tab.



Answer the questions below

In which tab is the disassembly view shown in x32dbg?

CPU tab

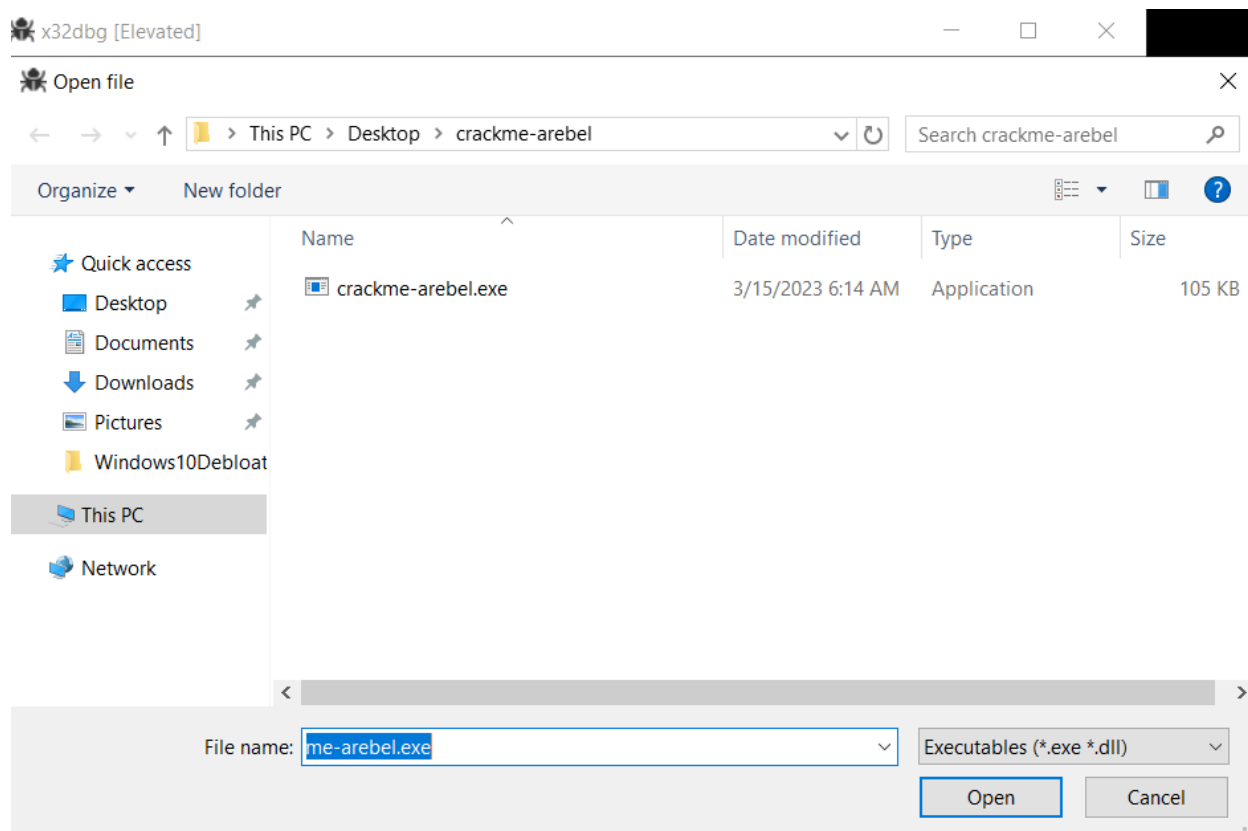
If a process opens a file or a process, where can we see information regarding that opened file or process?

Handles tab

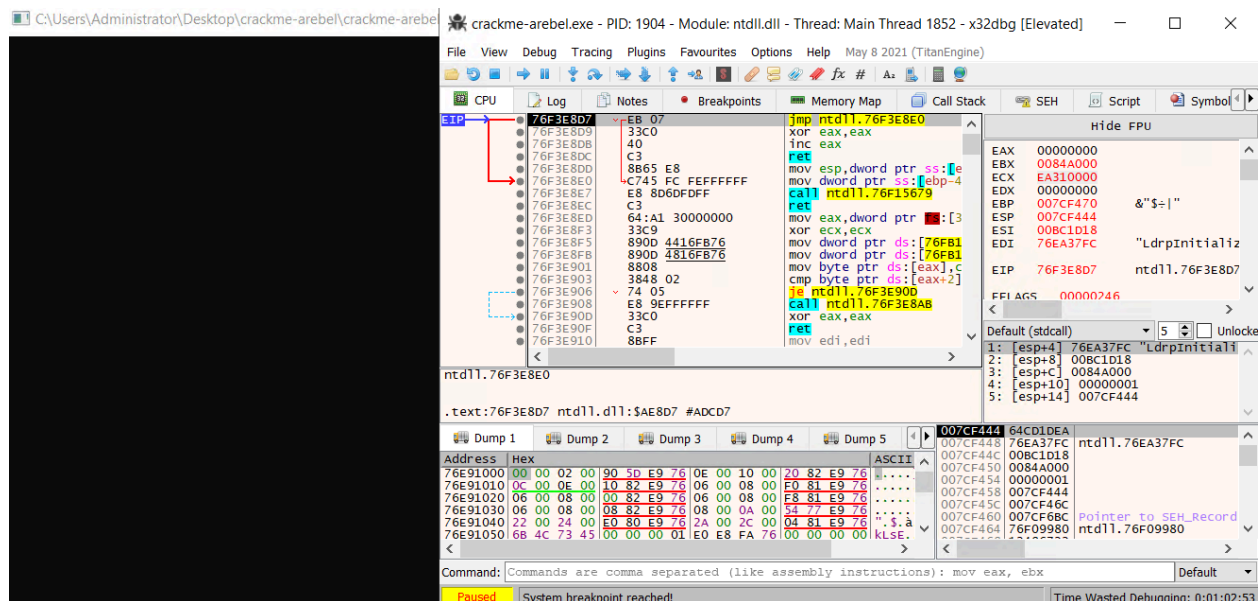
Task 5 - Debugging in Practice

x32dbg is an open-source, user-mode x86 assembly-level debugger for Microsoft Windows. It is part of the **x64dbg** project, where **x32dbg** handles 32-bit executables, and **x64dbg** handles 64-bit executables.

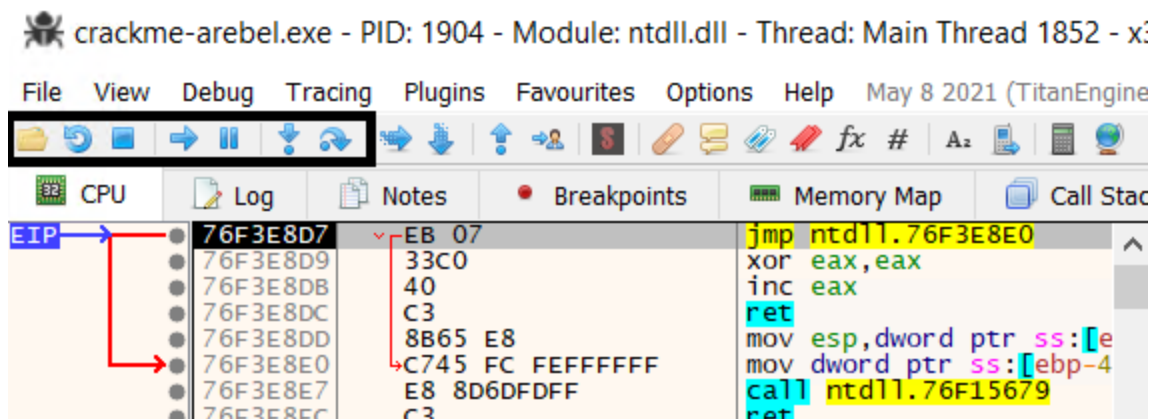
Open one of the crackmes from the attached VM. We can do this by clicking File > Open. We will see the below window asking us to select the file we want to select. Select crackme-arebel.exe as the file we want to debug.



The debugger attaches itself to the process and pauses it before it starts. We can see that a blank Command window opens in the background. This shows that the process has started but was stopped by the debugger. Please note that this window might not open with all processes. It will depend on the User Interface of the process. In the debugger window, we can see the familiar disassembly, the registers, the stack and other useful information. Please note that the address might not always be as shown in the images.



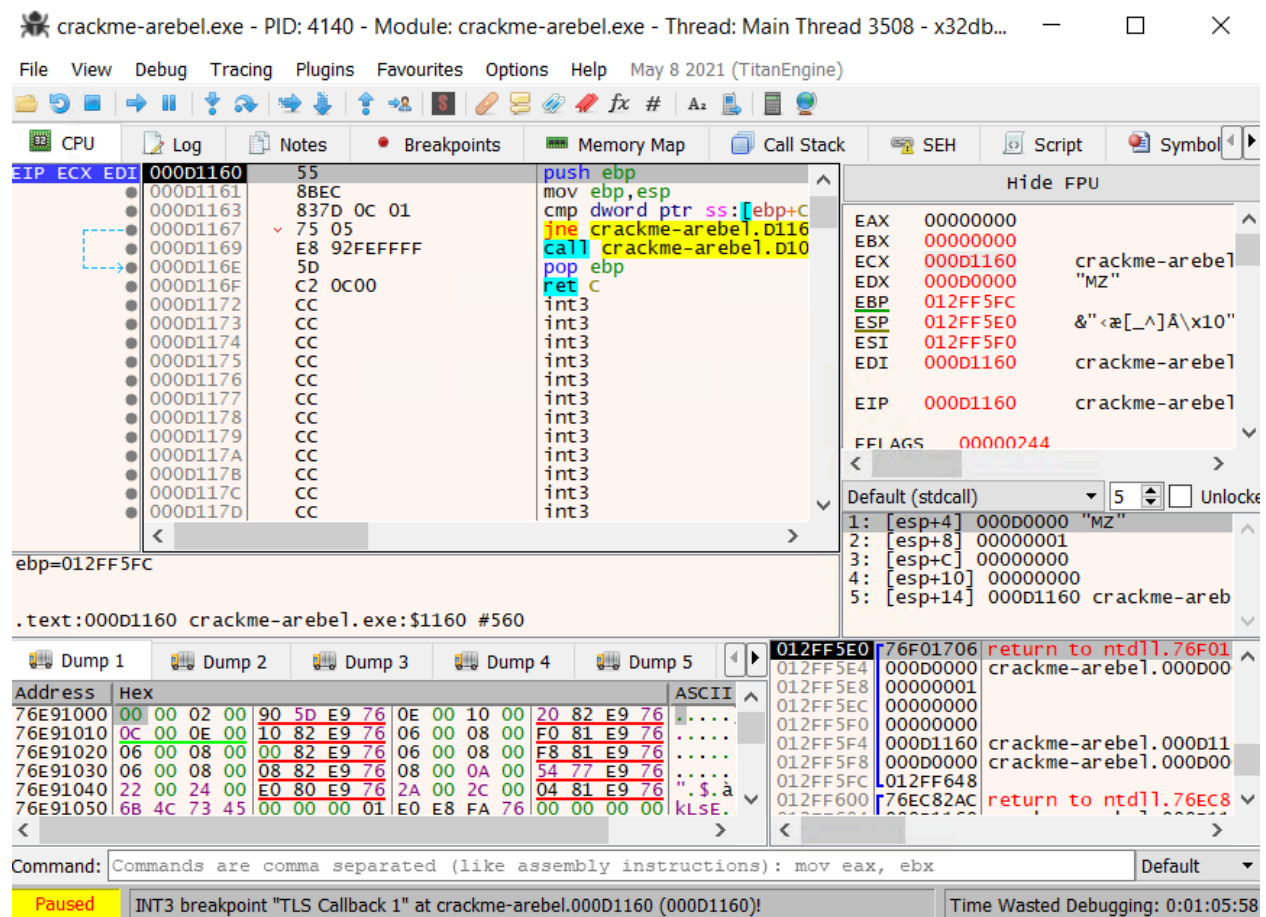
In the debugger window, we have some features that help us control the execution. In the below screenshot, we can see some of them highlighted.



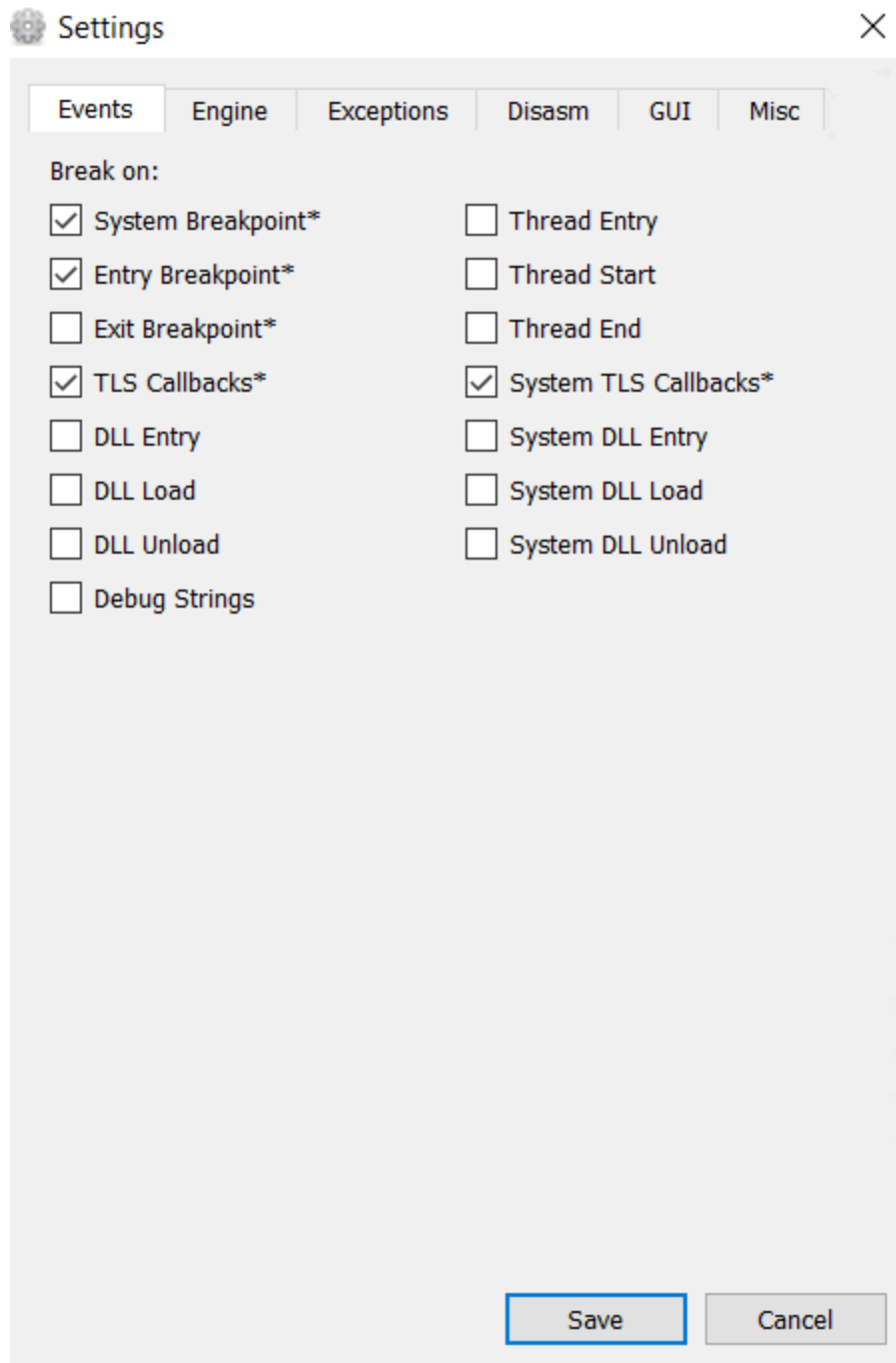
Among these, from left to right, we have the feature for opening a new file, restarting the execution of the opened file from the start, and stopping the execution. The arrow key will execute the program until it is stopped or paused by some other control.

So now that we know how to control the execution, let's execute the program using the arrow key (Please only press it once). When we do that, we can see in the bottom right corner that the program's status changes to Running, then Paused. Along with the status, we have the reason for pausing the execution. It says "INT3 breakpoint 'TLS Callback 1'...". This means that we have hit

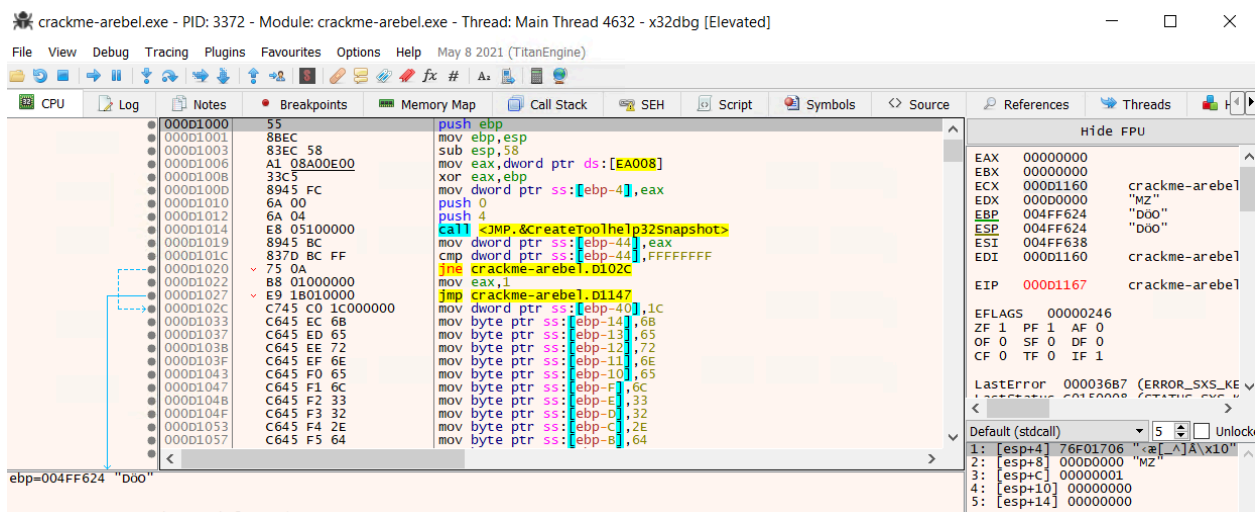
a TLS callback(opens in new tab), and the debugger was programmed to break execution on TLS callbacks.



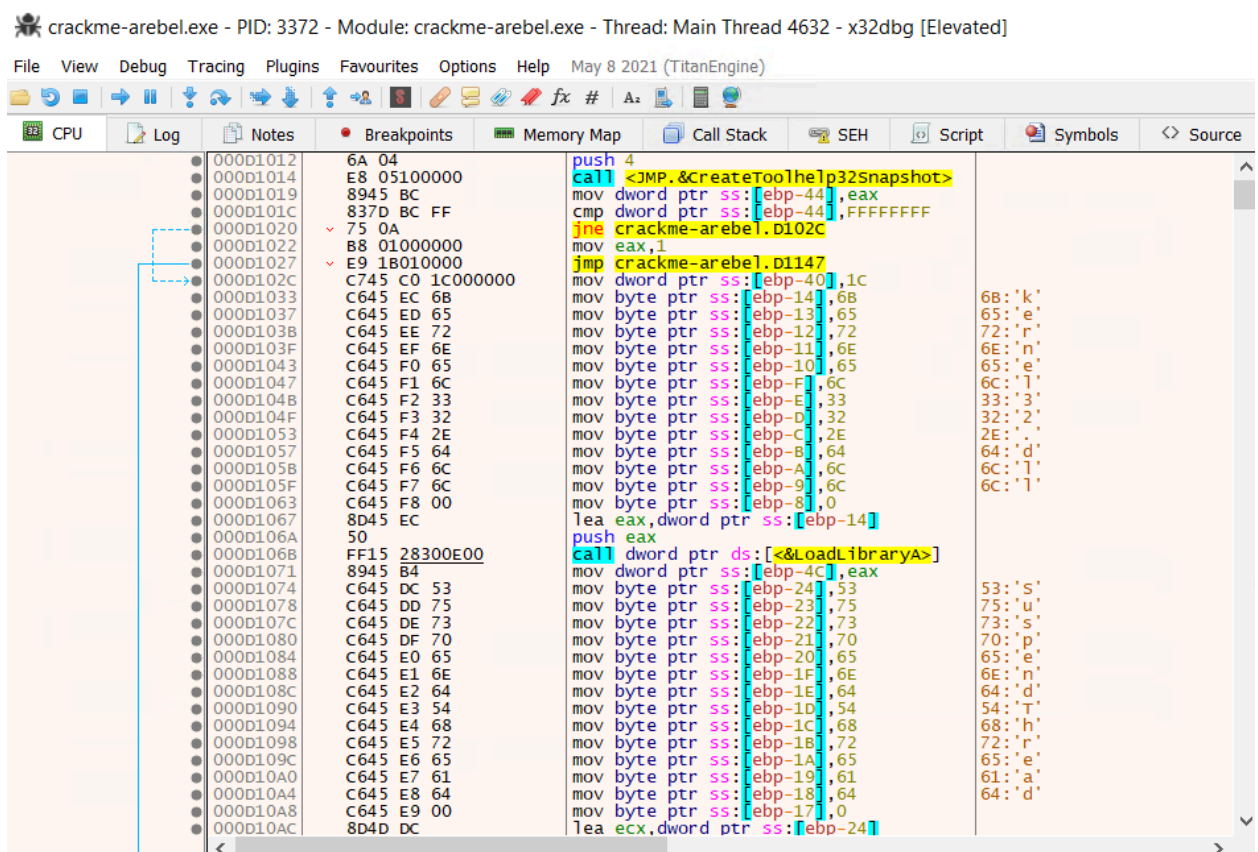
In the debugger, we can set where to put automatic breakpoints on a program by going to the Options > Preferences menu. The following screenshot shows the different points where breakpoints are automatically placed. We can see that TLS Callbacks and System TLS Callbacks are checked, which means execution will be broken on these events.



Since TLS callbacks are often used as an anti-reverse engineering technique, we should be careful when navigating this TLS callback. Therefore, it will be prudent to single-step each instruction while we are in the callback. We can do that by clicking the step-into option we discussed above. After stepping into every instruction, we see the EIP moving to the next instruction, and the values in the registers and stack change accordingly.



These libraries can be used to evade detection or for legitimate purposes, but we don't know that for now. So we can move along this path, and if we see a red flag, we just restart it again using the restart execution option.



The SuspendThread API is being loaded. This TLS callback will suspend the thread based on detecting a running process, such as a debugger (the CreateToolhelp32Snapshot API helps identify running processes). This is why the program will freeze if we proceed with the execution. We can verify this hypothesis by moving forward along this code path. On verification, we find that this is an evasion code path. Therefore, we would like to bypass it. For that, we would like to take the jump in the start of the TLS callback.

Answer the questions below

The attached VM has a crackme in the directory Desktop > crackme-arebel. In that crackme, there is a TLS callback with a conditional jump. Is this conditional jump taken?

No

What is the value of the Zero Flag in the above-mentioned conditional jump?

1

Which API call in the mentioned sample is used for enumerating running processes?

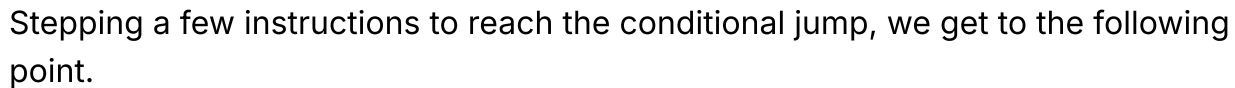
CreateToolhelp32Snapshot

From which Windows DLL is the API SuspendThread being called?

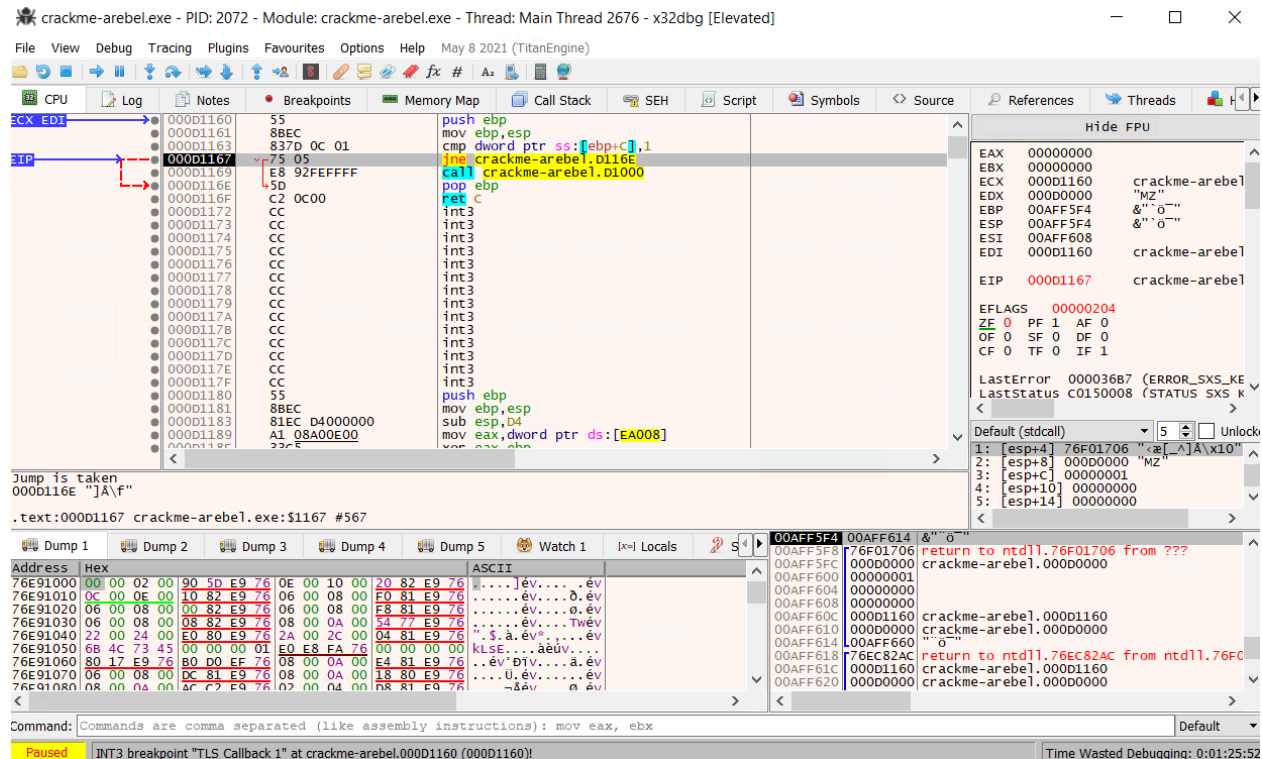
Kernel32.dll

Task 6 - Bypassing unwanted execution path

In the below screenshot, we can see that the execution has been restarted and brought back to the branch of code where we identified a detection evasion, that is, at the TLS callback breakpoint.



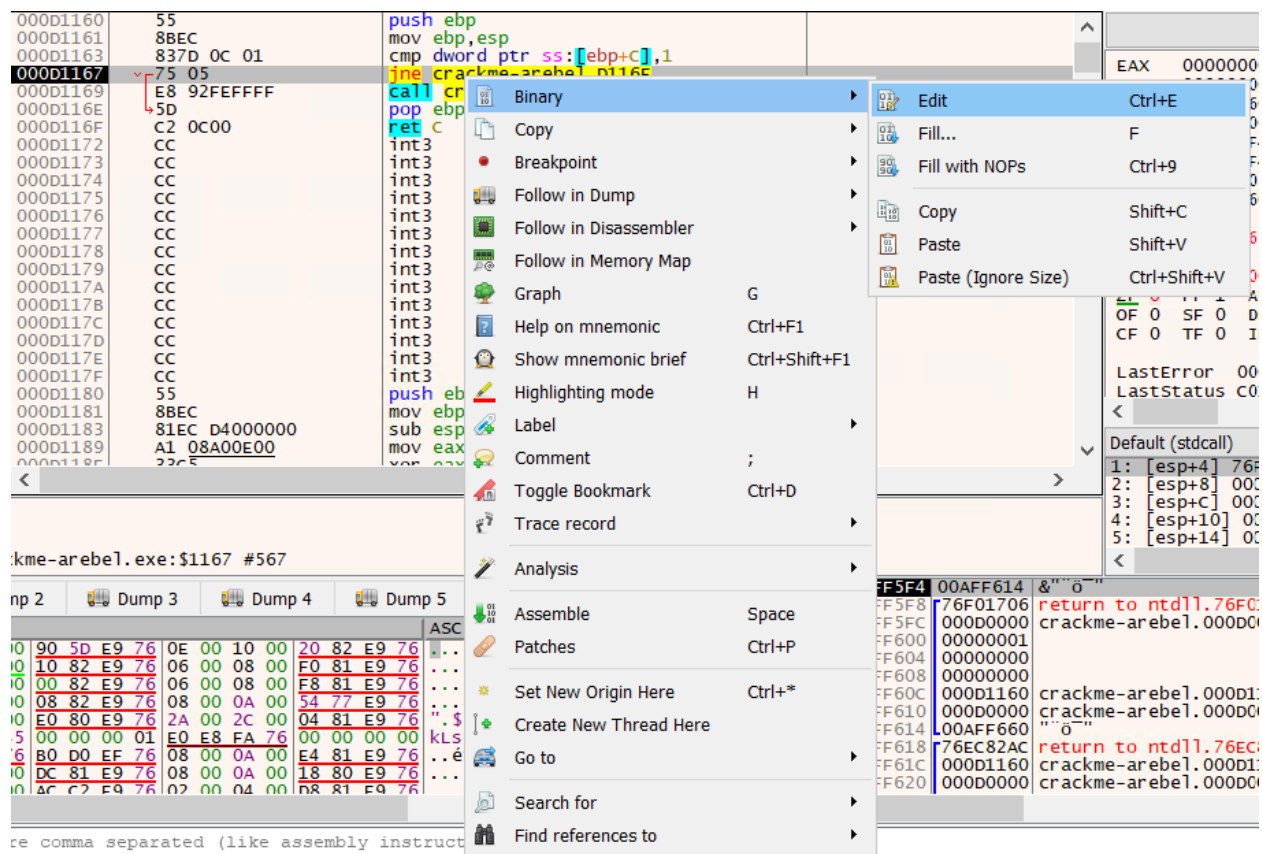
We see that the jump is not being taken at this point. In the previous task, we identified that the ZF was the reason for the jump not being taken. By double-clicking ZF in the right pane, we can change the ZF to 0.



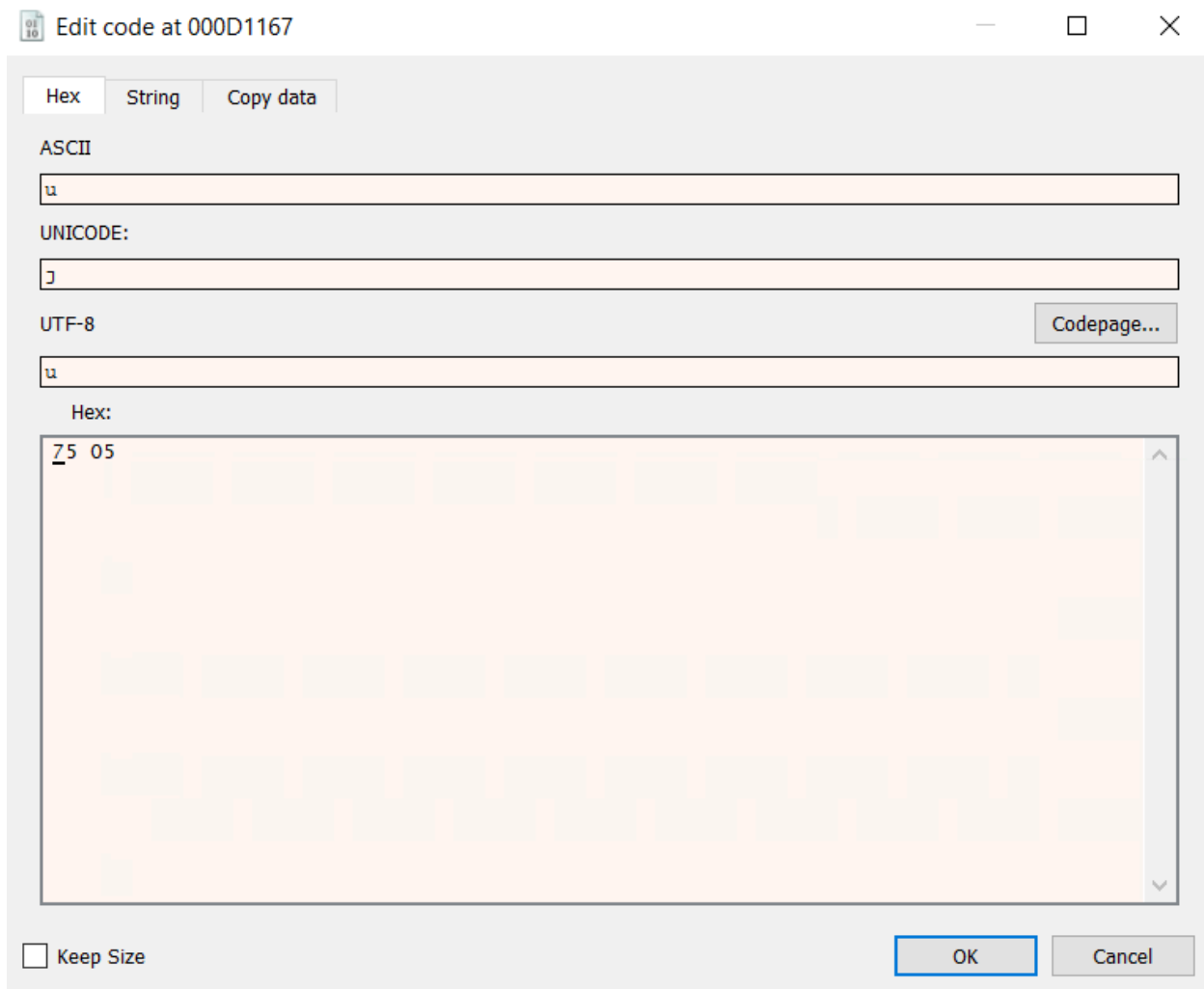
We can see here that the ZF is now 0, and the jump is taken. This way, we can bypass the unwanted execution path. If we step further or execute the program, it will take the jump and move to address D116E.

If we wish to defang the binary, we will have to change it so that the evasion path is not an option if it is just run through double-clicking. This is called patching. This way, after being defanged, we can perform dynamic analysis to reach our desired conclusion. Let's see how we can do that.

We can right-click on the instruction we want to edit to find the options related to that instruction. Among these is the option to edit, as seen in the following screenshot.



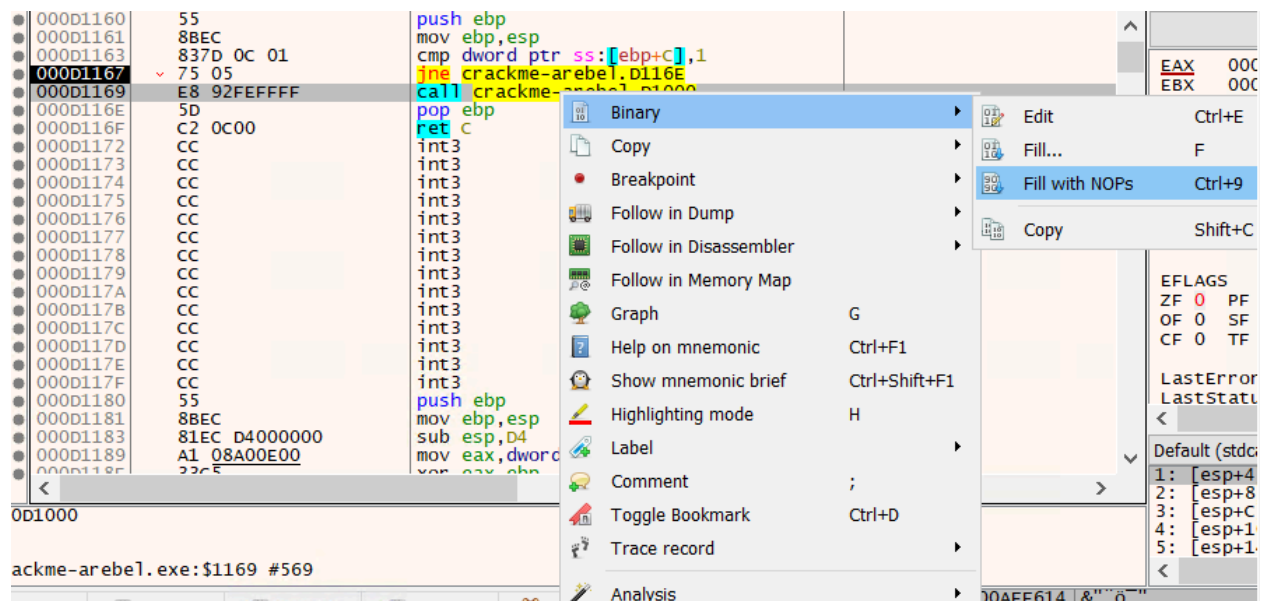
When we press edit, we see the following window.



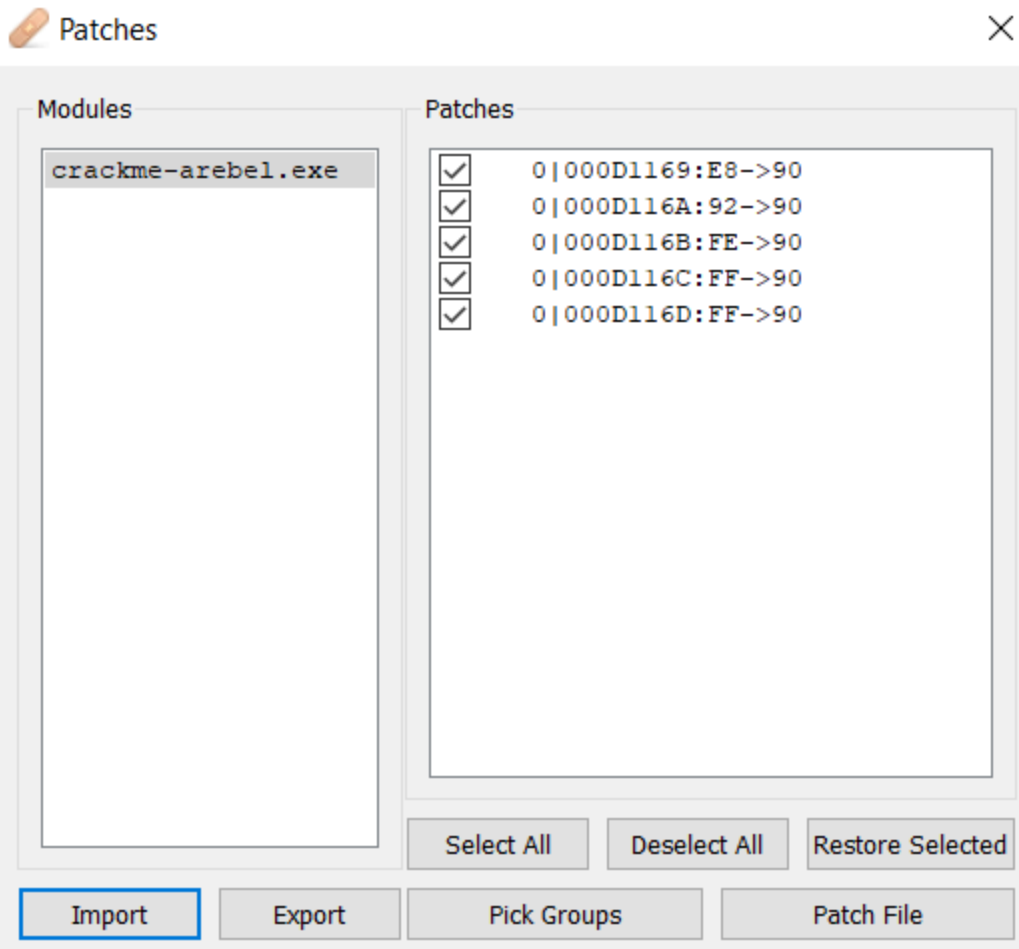
In the lowermost pane, we see 75 05 in hex, the opcode for the conditional jump instruction. We can change this in the following ways.

1. Change the jne to je. This way, the jump will be taken without us changing the ZF.
2. Change the conditional jump to an unconditional jump. This way, the jump will always be taken regardless of the condition above.

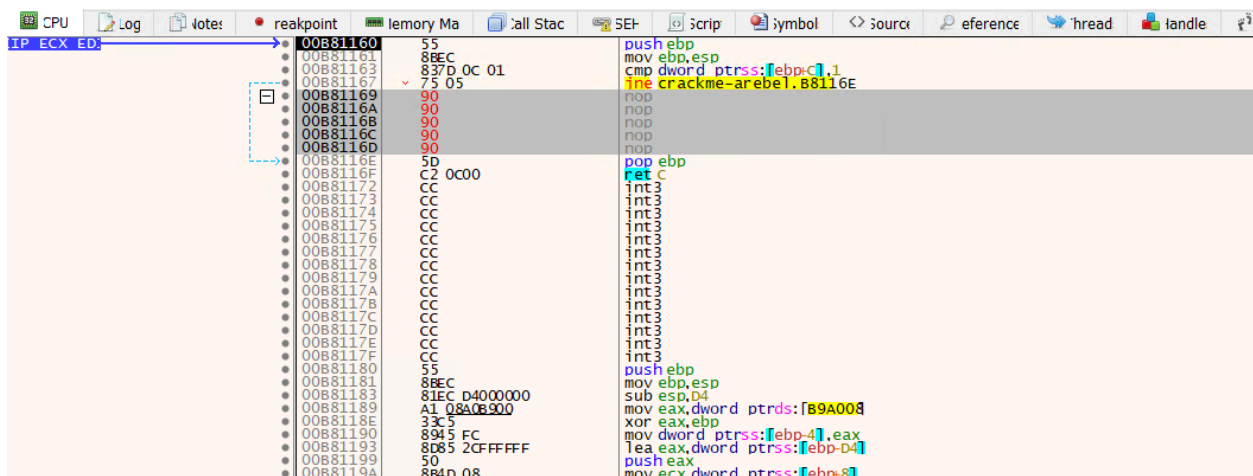
Among these two options, the second one is better as it gives us surety for the jump. However, there is one more way we can edit the binary for it to take our preferred execution path. That is, by filling the call instruction at address D1169 with NOPs. The screenshot below shows that we can fill the instruction with NOP by right-clicking and going to Binary > Fill with NOPs.

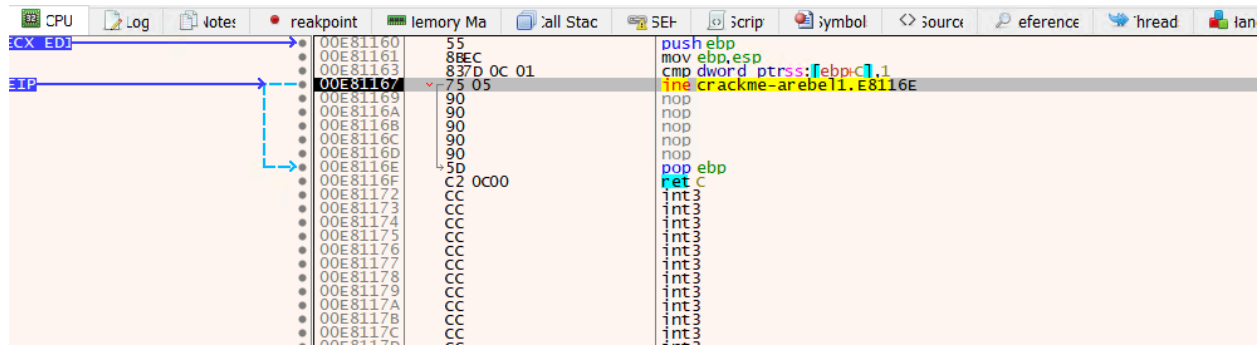


Once done, we can go to File > Patch File to write this file to disk. We might see something like the below window once we navigate to that.



By clicking Patch File on this window, the debugger will ask us where to save the patched file. This is how it will look once we have filled it with NOPs. This is more visible in the above screenshot, where each NOP instruction is mapped against equal sized Opcode.





Answer the questions below

What is it called when a binary's assembly code is permanently altered to get the desired execution path?

Patching

Task 7 - Conclusion

- Common techniques to evade static and basic dynamic malware analysis.
- The use of debuggers for deeper analysis of malware.
- Using debuggers for changing the environment at runtime.
- Using debuggers to patch malware samples.